

LINFO1131 Exam Jan. 2022

Guillaume Gillet

TOTAL POINTS

9.5 / 30

QUESTION 1

Limitations of declarative 10 pts

1.1 Lambda calculus 1 / 2

- 0 pts Correct
- ✓ - 1 pts Explanation is not formal
- 2 pts Incorrect or no answer

1.2 Communication channel 0 / 1

- 0 pts Correct
- 2 pts Incorrect or missing answer
- ✓ - 1 pts Partially correct

1.3 Client/server impossibility 0 / 1

- 0 pts Correct
- ✓ - 1 pts Incorrect or explanation incomplete

1.4 Port definition 0 / 2

- 0 pts Correct
- ✓ - 2 pts Incorrect or too incomplete
- 1 pts Incomplete

1.5 Port semantics 0 / 2

- 0 pts Correct
- ✓ - 2 pts Incorrect or too incomplete
- 0.5 pts Mostly correct with small error
- 1 pts Partly correct

1.6 Client/server with port 1 / 2

- 0 pts Correct
- 2 pts Incorrect or too incomplete
- ✓ - 1 pts Missing client send message
- 1 pts Missing server loop
- 0.5 pts Small error

QUESTION 2

2 Erlang supervisors 0 / 4

- 0 pts Correct
- 0.5 pts Supervisor tree partially correct
- ✓ - 1 pts Supervisor tree wrong
- 0.5 pts One_for_one partially correct
- ✓ - 1 pts One_for_one wrong
- 0.5 pts ETS partially correct
- ✓ - 1 pts ETS wrong
- 0.5 pts Linking partially correct
- ✓ - 1 pts Linking wrong

QUESTION 3

3 Reentrant locks 3 / 4

- 0 pts Correct
- 0.5 pts Specification partially correct
- 0.5 pts Reason for reentrancy partially correct
- 2 pts Implementation incorrect
- 0.5 pts Small error in implementation
- 1 pts Reentrancy incorrect
- 4 pts Incorrect
- ✓ - 1 pts Implementation partly correct
- 1.5 pts Implementation mostly incorrect
- 1 pts Specification incorrect

QUESTION 4

4 Monitors 0 / 4

- 0 pts Correct
- 1 pts Does not explain how wait set works
- 2 pts Pattern incorrect
- 1 pts Pattern partially correct (ex. uses if instead of while, or missing or incorrect semantic operation)
- 1 pts Semantic operation missing (ex. wait)
- ✓ - 4 pts Incorrect
- 2 pts Semantics missing
- 1.5 pts One small idea of the pattern

QUESTION 5

5 Deadlocks 1 / 2

- 0 pts Correct
- ✓ - 1 pts No definition of wait-for graph
- 1 pts Incorrect or missing example of wait-for graph
- 0.5 pts Partial definition of wait-for graph
- 0.5 pts Example wait-for graph is for another scenario

QUESTION 6

6 Liveness 1 / 2

- 0 pts Correct
- ✓ - 1 pts No formal definition or formal definition incorrect
- 1 pts Example incorrect or missing
- 0.5 pts Formal definition partially correct

QUESTION 7

7 Transaction manager 2.5 / 4

- 0 pts Correct
- 4 pts Incorrect
- 2 pts Pseudocode incorrect or missing
- ✓ - 1 pts Pseudocode partially correct
- 1.5 pts Small part of pseudocode correct
- 2 pts State diagram incorrect or missing
- 0.5 pts Most ideas of pseudocode correct
- 1 pts Part of state diagram correct
- ✓ - 0.5 pts Most of state diagram correct

Please write all answers inside the rectangles

Final exam LINFO1131

January 21, 2022
Prof. Peter Van Roy

First and last name GILLET GUILLAUME

NOMA 26361900

Q1: Limitations of declarative programming [10pts] (Q1 corresponds to the midterm)

1. Use lambda calculus to formally model declarative execution and use this to give a precise explanation of the limitations of pure declarative programming. [2pts]

La limite des programmes déclaratifs est le fait que le résultat doit toujours être attendu et "certain" dans les programmes tel que "client/server", "producteur/consommateur" ne peuvent être déterministe.

2. Explain formally how to add a communication channel (with FIFO property) to a lambda execution. Explain why this is no longer declarative. [1pt]

⇒ Ce n'est plus déclaratif car en ajoutant un canal de communication, les données arrivent dans un ordre déterminé (à cause de la propriété FIFO).

3. Explain why it is impossible to implement a purely declarative client/server application that satisfies the requirement of client liveness, namely that the response time for each client request depends only on the message travel time of the request and the server computation time for handling the request. [1pt]

Il est impossible de l'implémenter car si deux clients font une requête en même temps, une sera privilégiée par rapport à l'autre car notre serveur ne peut pas "calculer" les 2 requêtes en même temps. Et de plus, si un client est (dans notre code) est prioritaire par rapport à un autre et que celui-ci envoie des requêtes non-stop, l'autre client ne sera jamais servi.

⇒ En cela un serveur peut être "déclaratif" et satisfaire les "clients libres". Si celui-ci peut avoir un "response time" plus grand que celui dans l'énoncé.

4. Define the concept of a port in Oz, which is a form of communication channel, and give a code example of how it executes using NewPort and Send. [2pts]

Un port en Oz est comme un "cable" / "réseau" entre des Variables / Serveurs.

$$x = \{ \text{NewPort}(\tau, \text{Id}) \}$$

$$\{ \text{Handle } x \}$$

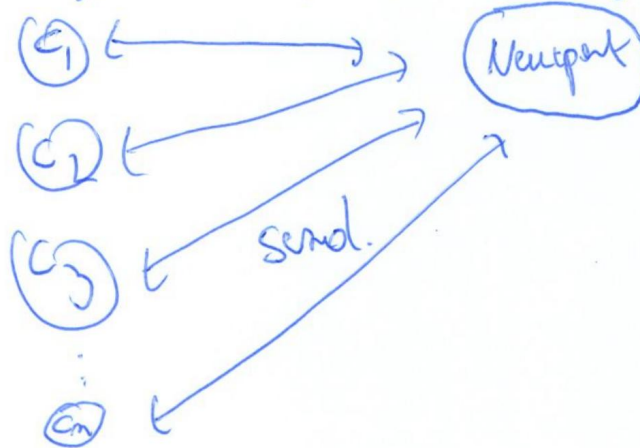
$$\{ \text{Send}(\tau, \text{Id}) \}$$

First and last name Gilbert Gulluon

NOMA 26361800.

5. Give the formal semantics of ports, for both the NewPort and Send operations. [2pts]

un port reçoit des informations et peut en Renvoyer aux clients, les Stocker, ... (il est la porte d'entrée des liens).



6. Explain how to implement a client/server application that satisfies the requirement of client liveness, using exactly one port and where the rest of the application is purely declarative. Give the code of the client and server in Oz using a port. [2pts]

on attend les Request qui arrivent en même temps puis on les exécute :

exemple pour 2 clients :

on aie notre port puis on écrit ceci :

$Z = \{waitTwo\ x_1, x_2\}$

case $Z \mid S_1 \mid S_2$ of

$\mid (M_1, T_1) \mid S_2$ Then $\{Handle\ M_1\} \{Send\ T_1 \mid S_2\}$

$\mid (M_2, T_2) \mid S_1$ Then $\{Handle\ M_2\} \{Send\ T_2 \mid S_1\}$

end

First and last name *W/lt Guillaume*

NOMA *26361800*

Q2: Erlang supervisors [4pts]

For this question, answer the following points:

1. Define what is a supervisor tree and explain what it does. [1pt]
2. Define what is one_for_one supervision and give an example where it is needed. [1pt]
3. Explain what is ETS and why it is important for a supervisor. [1pt]
4. Define process linking and explain how it is used to implement supervisors. [1pt]

1. Un arbre qui gère par priorité les différents éléments à l'intérieur de celui-ci.

First and last name *Gillet Guillaume*

NOMA *20361300*

Q3: Reentrant locks [4 pts]

For this question, answer the following three points:

1. Specify a reentrant lock. [1pt]
2. Explain why reentrancy is important. Give a scenario where it is needed. [1pt]
3. Give an implementation of the reentrant lock, similar to what we saw in the course. [2pts]

1. Le reentrant lock est comme un lock classique (une fonction qui est entouré d'un lock pour n'être exécuté que par un thread à la fois). Sauf que celui-ci gère le problème que si notre fonction dans le lock reçoit appelée à notre lock le système n'attendra pas le "unlock" indéfiniment.

2. "Reentrancy" est donc important pour éviter un deadlock, que notre fonction attende indéfiniment un unlock.

3. Reentrant lock:

```
if ctr = {Thread.This} Then
    <P> → fonction qui peut faire
    appelé à reentrant lock.
else
    Lock.
    ctr = {Thread.This}
    <P>.
end.
ctr = Unit.
end.
```

First and last name	Gilad Goltz
NOMA	26 36 1800

Q4: Monitors [4pts]

For this question, answer the following two points:

1. Give the semantics of monitors. [2pts]
2. Give a programming pattern for monitors that guarantees correctness. [2pts]

First and last name Gilbert Guillaume

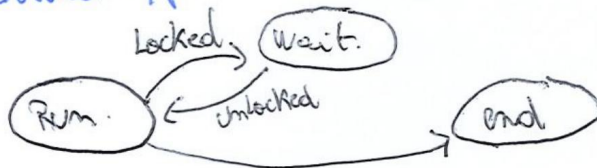
NOMA 26761800.

Q5: Deadlocks [2pts]

Give a formal definition of deadlock using the wait-for graph. As part of your answer, define the wait-for graph. Give an example of a deadlock with three transactions, T1, T2, and T3, where T1 uses cells C2 and C3, T2 uses C3 and C1, and T3 uses C1 and C2.

deadlock: quand un Thread attend indéfiniment la libération d'un lock (unlock).

exemple: $T_1 \text{ use } C_3 \xrightarrow{\text{then}} T_2 \text{ use } C_1 \xrightarrow{\text{then}} T_3 \text{ use } C_2$ mtn les threads sont bloqués car C_3, C_2, C_1 sont lock et T_1 attend C_2 pour la terminer, T_2 attends C_3 pour la terminer et T_3 attends C_1 pour la terminer.



Q6: Liveness [2pts]

Give a formal definition of what is a liveness property and give an example of a liveness property.

Liveness: "at least" le programme s'exécute pour au moins les propriétés observées (un peu opposé à Safety).

exemple: le client envoie un message.

↳ liveness: le client envoie au moins 1 message $\Rightarrow$ "at least"
↳ Safety: le client envoie au plus 1 message $\Rightarrow$ "at most"

First and last name Elit Guillem

NOMA 26361800.

Q7: Transaction manager [4pts]

For this question answer the following two points:

1. Give a pseudocode definition of a transaction manager that implements deadlock avoidance. Explain how your definition avoids deadlocks. [2pts]

2. Give a state diagram for the transaction manager. You can give either the state diagram for the first algorithm or for the improved algorithm. [2pts]

1. Priority = Assigne Valeurs $<$ Tout les autres priority deja existant (les autres threads).
- On essaye de rentrer dans le Lock.
 - ↳ Si on peut on y va (Run).
 - ↳ Sinon ?
 - ↳ Si priority $>$ priority du Thread qui Lock^(T₂) on sort le Thread (T₂) et on rentre (Run).
 - ↳ Sinon wait. unlock.
 - Après avoir finis le Run on Unlock.

2.

